

cases worth distinguishing are: (i) The curve is an open curve, so that $\mathbf{x}_0$ and $\mathbf{x}_{N-1}$ represent endpoints. (ii) The curve is a closed curve, so that there is an implied curve segment connecting $\mathbf{x}_{N-1}$ back to $\mathbf{x}_0$.

A straightforward solution, using methods already at hand, is first to approximate distance along the curve by the sum of chord lengths between the tabulated points, and then to construct spline interpolations for each of the coordinates, $0, \dots, n-1$, as a function of that parameter. Since the derivative of any single coordinate with respect to arc length can be no greater than 1, it is guaranteed that the spline interpolations will be well-behaved.

Probably 90% of applications require nothing more complicated than the above. If you are in the unhappy 10%, then you will need to learn about *Bézier curves*, *B-splines*, and *interpolating splines* more generally [12,13,14]. For the happy majority, an implementation is

interp_curve.h

```
struct Curve_interp {
    Object for interpolating a curve specified by n points in dim dimensions.
    Int dim, n, in;
    Bool cls;                               Set if a closed curve.
    MatDoub pts;
    VecDoub s;
    VecDoub ans;
    NRvector<Spline_interp*> srp;

    Curve_interp(MatDoub &ptsin, Bool close=0)
    : n(ptsin.nrows()), dim(ptsin.ncols()), in(close ? 2*n : n),
      cls(close), pts(dim,in), s(in), ans(dim), srp(dim) {
        Constructor. The n × dim matrix ptsin inputs the data points. Input close as 0 for
        an open curve, 1 for a closed curve. (For a closed curve, the last data point should not
        duplicate the first — the algorithm will connect them.)
        Int i, ii, im, j, ofs;
        Doub ss, soff, db, de;
        ofs = close ? n/2 : 0;               The trick for closed curves is to duplicate half a
        s[0] = 0.;                           period at the beginning and end, and then
        for (i=0; i<in; i++) {               use the middle half of the resulting spline.
            ii = (i-ofs+n) % n;
            im = (ii-1+n) % n;
            for (j=0; j<dim; j++) pts[j][i] = ptsin[ii][j];   Store transpose.
            if (i>0) {                       Accumulate arc length.
                s[i] = s[i-1] + rad(&ptsin[ii][0], &ptsin[im][0]);
                if (s[i] == s[i-1]) throw("error in Curve_interp");
                Consecutive points may not be identical. For a closed curve, the last data
                point should not duplicate the first.
            }
        }
        ss = close ? s[ofs+n]-s[ofs] : s[n-1]-s[0]; Rescale parameter so that the
        soff = s[ofs];                             interval [0,1] is the whole curve (or one period).
        for (i=0; i<in; i++) s[i] = (s[i]-soff)/ss;
        for (j=0; j<dim; j++) {               Construct the splines using endpoint derivatives.
            db = in < 4 ? 1.e99 : fprime(&s[0], &pts[j][0], 1);
            de = in < 4 ? 1.e99 : fprime(&s[in-1], &pts[j][in-1], -1);
            srp[j] = new Spline_interp(s, &pts[j][0], db, de);
        }
    }
    ~Curve_interp() {for (Int j=0; j<dim; j++) delete srp[j];}

    VecDoub &interp(Doub t) {
        Interpolate a point on the stored curve. The point is parameterized by t, in the range [0,1].
        For open curves, values of t outside this range will return extrapolations (dangerous!). For
        closed curves, t is periodic with period 1.
    }
};
```

```

    if (cls) t = t - floor(t);
    for (Int j=0;j<dim;j++) ans[j] = (*srp[j]).interp(t);
    return ans;
}

Doub fprime(Doub *x, Doub *y, Int pm) {
    Utility for estimating the derivatives at the endpoints. x and y point to the abscissa and
    ordinate of the endpoint. If pm is +1, points to the right will be used (left endpoint); if it
    is -1, points to the left will be used (right endpoint). See text, below.
    Doub s1 = x[0]-x[pm*1], s2 = x[0]-x[pm*2], s3 = x[0]-x[pm*3],
        s12 = s1-s2, s13 = s1-s3, s23 = s2-s3;
    return -(s1*s2/(s13*s23*s3))*y[pm*3]+(s1*s3/(s12*s2*s23))*y[pm*2]
        -(s2*s3/(s1*s12*s13))*y[pm*1]+(1./s1+1./s2+1./s3)*y[0];
}

Doub rad(const Doub *p1, const Doub *p2) {
    Euclidean distance.
    Doub sum = 0.;
    for (Int i=0;i<dim;i++) sum += SQR(p1[i]-p2[i]);
    return sqrt(sum);
}

};

```

The utility routine `fprime` estimates the derivative of a function at a tabulated abscissa x_0 using four consecutive tabulated abscissa-ordinate pairs, $(x_0, y_0), \dots, (x_3, y_3)$. The formula for this, readily derived by power-series expansion, is

$$y'_0 = -C_0 y_0 + C_1 y_1 - C_2 y_2 + C_3 y_3 \quad (3.7.17)$$

where

$$\begin{aligned}
 C_0 &= \frac{1}{s_1} + \frac{1}{s_2} + \frac{1}{s_3} \\
 C_1 &= \frac{s_2 s_3}{s_1 (s_2 - s_1) (s_3 - s_1)} \\
 C_2 &= \frac{s_1 s_3}{(s_2 - s_1) s_2 (s_3 - s_2)} \\
 C_3 &= \frac{s_1 s_2}{(s_3 - s_1) (s_3 - s_2) s_3}
 \end{aligned} \quad (3.7.18)$$

with

$$\begin{aligned}
 s_1 &\equiv x_1 - x_0 \\
 s_2 &\equiv x_2 - x_0 \\
 s_3 &\equiv x_3 - x_0
 \end{aligned} \quad (3.7.19)$$

CITED REFERENCES AND FURTHER READING:

- Buhmann, M.D. 2003, *Radial Basis Functions: Theory and Implementations* (Cambridge, UK: Cambridge University Press).[1]
- Powell, M.J.D. 1992, "The Theory of Radial Basis Function Approximation" in *Advances in Numerical Analysis II: Wavelets, Subdivision Algorithms and Radial Functions*, ed. W. A. Light (Oxford: Oxford University Press), pp. 105–210.[2]
- Wikipedia. 2007+, "Radial Basis Functions," at <http://en.wikipedia.org/>.[3]
- Beatson, R.K. and Greengard, L. 1997, "A Short Course on Fast Multipole Methods", in *Wavelets, Multilevel Methods and Elliptic PDEs*, eds. M. Ainsworth, J. Levesley, W. Light, and M. Marletta (Oxford: Oxford University Press), pp. 1–37.[4]

- Beatson, R.K. and Newsam, G.N. 1998, "Fast Evaluation of Radial Basis Functions: Moment-Based Methods" in *SIAM Journal on Scientific Computing*, vol. 19, pp. 1428-1449.[5]
- Wendland, H. 2005, *Scattered Data Approximation* (Cambridge, UK: Cambridge University Press).[6]
- Franke, R. 1982, "Scattered Data Interpolation: Tests of Some Methods," *Mathematics of Computation*, vol. 38, pp. 181-200.[7]
- Shepard, D. 1968, "A Two-dimensional Interpolation Function for Irregularly-spaced Data," in *Proceedings of the 1968 23rd ACM National Conference* (New York: ACM Press), pp. 517-524.[8]
- Cressie, N. 1991, *Statistics for Spatial Data* (New York: Wiley).[9]
- Wackernagel, H. 1998, *Multivariate Geostatistics*, 2nd ed. (Berlin: Springer).[10]
- Rybbicki, G.B., and Press, W.H. 1992, "Interpolation, Realization, and Reconstruction of Noisy, Irregularly Sampled Data," *Astrophysical Journal*, vol. 398, pp. 169-176.[11]
- Isaaks, E.H., and Srivastava, R.M. 1989, *Applied Geostatistics* (New York: Oxford University Press).
- Deutsch, C.V., and Journel, A.G. 1992, *GSLIB: Geostatistical Software Library and User's Guide* (New York: Oxford University Press).
- Knott, G.D. 1999, *Interpolating Cubic Splines* (Boston: Birkhäuser).[12]
- De Boor, C. 2001, *A Practical Guide to Splines* (Berlin: Springer).[13]
- Prautzsch, H., Boehm, W., and Paluszny, M. 2002, *Bézier and B-Spline Techniques* (Berlin: Springer).[14]

3.8 Laplace Interpolation

In this section we look at a *missing data* or *gridding* problem, namely, how to restore missing or unmeasured values on a regular grid. Evidently some kind of interpolation from the not-missing values is required, but how shall we do this in a principled way?

One good method, already in use at the dawn of the computer age [1,2], is *Laplace interpolation*, sometimes called *Laplace/Poisson interpolation*. The general idea is to find an interpolating function y that satisfies Laplace's equation in n dimensions,

$$\nabla^2 y = 0 \quad (3.8.1)$$

wherever there is no data, and which satisfies

$$y(\mathbf{x}_i) = y_i \quad (3.8.2)$$

at all measured data points. Generically, such a function does exist. The reason for choosing Laplace's equation (among all possible partial differential equations, say) is that the solution to Laplace's equation selects, in some sense, the smoothest possible interpolant. In particular, its solution minimizes the integrated square of the gradient,

$$\int_{\Omega} |\nabla y|^2 d\Omega \quad (3.8.3)$$

where Ω denotes the n -dimensional domain of interest. This is a very general idea, and it can be applied to irregular meshes as well as to regular grids. Here, however, we consider only the latter.

For purposes of illustration (and because it is the most useful example) we further specialize to the case of two dimensions, and to the case of a Cartesian grid whose x_1 and x_2 values are evenly spaced — like a checkerboard.

In this geometry, the finite difference approximation to Laplace’s equation has a particularly simple form, one that echos the *mean value theorem* for continuous solutions of the Laplace equation: The value of the solution at any free gridpoint (i.e., not a point with a measured value) equals the average of its four Cartesian neighbors. (See §20.0.) Indeed, this already sounds a lot like interpolation.

If y_0 denotes the value at a free point, while y_u , y_d , y_l , and y_r denote the values at its up, down, left, and right neighbors, respectively, then the equation satisfied is

$$y_0 - \frac{1}{4}y_u - \frac{1}{4}y_d - \frac{1}{4}y_l - \frac{1}{4}y_r = 0 \quad (3.8.4)$$

For gridpoints with measured values, on the other hand, a different (simple) equation is satisfied,

$$y_0 = y_{0(\text{measured})} \quad (3.8.5)$$

Note that these nonzero right-hand sides are what make an inhomogeneous, and therefore generally solvable, set of linear equations.

We are not quite done, since we must provide special forms for the top, bottom, left, and right boundaries, and for the four corners. Homogeneous choices that embody “natural” boundary conditions (with no preferred function values) are

$$\begin{aligned} y_0 - \frac{1}{2}y_u - \frac{1}{2}y_d &= 0 && \text{(left and right boundaries)} \\ y_0 - \frac{1}{2}y_l - \frac{1}{2}y_r &= 0 && \text{(top and bottom boundaries)} \\ y_0 - \frac{1}{2}y_r - \frac{1}{2}y_d &= 0 && \text{(top-left corner)} \\ y_0 - \frac{1}{2}y_l - \frac{1}{2}y_d &= 0 && \text{(top-right corner)} \\ y_0 - \frac{1}{2}y_r - \frac{1}{2}y_u &= 0 && \text{(bottom-left corner)} \\ y_0 - \frac{1}{2}y_l - \frac{1}{2}y_u &= 0 && \text{(bottom-right corner)} \end{aligned} \quad (3.8.6)$$

Since every gridpoint corresponds to exactly one of the equations in (3.8.4), (3.8.5), or (3.8.6), we have exactly as many equations as there are unknowns. If the grid is M by N , then there are MN of each. This can be quite a large number; but the equations are evidently very sparse. We solve them by defining a derived class from §2.7’s `Linbcg` base class. You can readily identify all the cases of equations (3.8.4) – (3.8.6) in the code for `atimes`, below.

```
struct Laplace_interp : Linbcg {
```

[interp_laplace.h](#)

Object for interpolating missing data in a matrix by solving Laplace’s equation. Call constructor once, then solve one or more times (see text).

```
    MatDoub &mat;
    Int ii,jj;
    Int nn,iter;
    VecDoub b,y,mask;
```

```
    Laplace_interp(MatDoub_IO &matrix) : mat(matrix), ii(mat.nrows()),
    jj(mat.ncols()), nn(ii*jj), iter(0), b(nn), y(nn), mask(nn) {
```

Constructor. Values greater than $1.e99$ in the input matrix `mat` are deemed to be missing data. The matrix is not altered until `solve` is called.

```
        Int i,j,k;
        Doub vl = 0.;
```

```

for (k=0;k<nn;k++) {
    i = k/jj;
    j = k - i*jj;
    if (mat[i][j] < 1.e99) {
        b[k] = y[k] = v1 = mat[i][j];
        mask[k] = 1;
    } else {
        b[k] = 0.;
        y[k] = v1;
        mask[k] = 0;
    }
}
}

```

Fill the r.h.s. vector, the initial guess, and a mask of the missing data.

```

void asolve(VecDoub_I &b, VecDoub_O &x, const Int itrnsp);
void atimes(VecDoub_I &x, VecDoub_O &r, const Int itrnsp);

```

See definitions below. These are the real algorithmic content.

```

Doub solve(Doub tol=1.e-6, Int itmax=-1) {

```

Invoke Linbcg::solve with appropriate arguments. The default argument values will usually work, in which case this routine need be called only once. The original matrix mat is refilled with the interpolated solution.

```

    Int i,j,k;
    Doub err;
    if (itmax <= 0) itmax = 2*MAX(ii,jj);
    Linbcg::solve(b,y,1,tol,itmax,iter,err);
    for (k=0,i=0;i<ii;i++) for (j=0;j<jj;j++) mat[i][j] = y[k++];
    return err;
}
};

```

```

void Laplace_interp::asolve(VecDoub_I &b, VecDoub_O &x, const Int itrnsp) {

```

Diagonal preconditioner. (Diagonal elements all unity.)

```

    Int i,n=b.size();
    for (i=0;i<n;i++) x[i] = b[i];
}

```

```

void Laplace_interp::atimes(VecDoub_I &x, VecDoub_O &r, const Int itrnsp) {

```

Sparse matrix, and matrix transpose, multiply. This routine embodies eqs. (3.8.4), (3.8.5), and (3.8.6).

```

    Int i,j,k,n=r.size(),jjt,it;
    Doub del;
    for (k=0;k<n;k++) r[k] = 0.;
    for (k=0;k<n;k++) {

```

```

        i = k/jj;
        j = k - i*jj;
        if (mask[k]) {
            r[k] += x[k];

```

Measured point, eq. (3.8.5).

```

        } else if (i>0 && i<ii-1 && j>0 && j<jj-1) {
            if (itrnsp) {
                r[k] += x[k];
                del = -0.25*x[k];
                r[k-1] += del;
                r[k+1] += del;
                r[k-jj] += del;
                r[k+jj] += del;
            } else {

```

Interior point, eq. (3.8.4).

```

                r[k] = x[k] - 0.25*(x[k-1]+x[k+1]+x[k+jj]+x[k-jj]);
            }
        }

```

```

    } else if (i>0 && i<ii-1) {
        if (itrnsp) {
            r[k] += x[k];
            del = -0.5*x[k];
            r[k-jj] += del;

```

Left or right edge, eq. (3.8.6).

```

        r[k+jj] += del;
    } else {
        r[k] = x[k] - 0.5*(x[k+jj]+x[k-jj]);
    }
} else if (j>0 && j<jj-1) {
    if (itrnsp) {
        r[k] += x[k];
        del = -0.5*x[k];
        r[k-1] += del;
        r[k+1] += del;
    } else {
        r[k] = x[k] - 0.5*(x[k+1]+x[k-1]);
    }
} else {
    jjt = i==0 ? jj : -jj;
    it = j==0 ? 1 : -1;
    if (itrnsp) {
        r[k] += x[k];
        del = -0.5*x[k];
        r[k+jjt] += del;
        r[k+it] += del;
    } else {
        r[k] = x[k] - 0.5*(x[k+jjt]+x[k+it]);
    }
}
}
}

```

Top or bottom edge, eq. (3.8.6).

Corners, eq. (3.8.6).

Usage is quite simple. Just fill a matrix with function values where they are known, and with 1.e99 where they are not; send the matrix to the constructor; and call the solve routine. The missing values will be interpolated. The default arguments should serve for most cases.

```

Int m=...,n=...;
MatDoub mat(m,n);
...
Laplace_interp mylaplace(mat);
mylaplace.solve();

```

Quite decent results are obtained for smooth functions on 300×300 matrices in which a random 10% of gridpoints have known function values, with 90% interpolated. However, since compute time scales as $MN \max(M, N)$ (that is, as the cube), this is not a method to use for much larger matrices, unless you break them up into overlapping tiles. If you experience convergence difficulties, then you should call solve, with appropriate nondefault arguments, several times in succession, and look at the returned error estimate after each call returns.

3.8.1 Minimum Curvature Methods

Laplace interpolation has a tendency to yield cone-like cusps around any small islands of known data points that are surrounded by a sea of unknowns. The reason is that, in two dimensions, the solution of Laplace's equation near a point source is logarithmically singular. When the known data is spread fairly evenly (if randomly) across the grid, this is not generally a problem. *Minimum curvature methods* deal with the problem at a more fundamental level by being based on the biharmonic equation

$$\nabla(\nabla y) = 0 \quad (3.8.7)$$

instead of Laplace's equation. Solutions of the biharmonic equation minimize the integrated square of the curvature,

$$\int_{\Omega} |\nabla^2 y|^2 d\Omega \quad (3.8.8)$$

Minimum curvature methods are widely used in the earth-science community [3,4].

The references give a variety of other methods that can be used for missing data interpolation and gridding.

CITED REFERENCES AND FURTHER READING:

- Noma, A.A. and Misulia, M.G. 1959, "Programming Topographic Maps for Automatic Terrain Model Construction," *Surveying and Mapping*, vol. 19, pp. 355–366.[1]
- Crain, I.K. 1970, "Computer Interpolation and Contouring of Two-dimensional Data: a Review," *Geoexploration*, vol. 8, pp. 71–86.[2]
- Burrough, P.A. 1998, *Principles of Geographical Information Systems*, 2nd ed. (Oxford, UK: Clarendon Press)
- Watson, D.F. 1982, *Contouring: A Guide to the Analysis and Display of Spatial Data* (Oxford, UK: Pergamon).
- Briggs, I.C. 1974, "Machine Contouring Using Minimum Curvature," *Geophysics*, vol. 39, pp. 39–48.[3]
- Smith, W.H.F. and Wessel, P. 1990, "Gridding With Continuous Curvature Splines in Tension," *Geophysics*, vol. 55, pp. 293–305.[4]